

Collaborative Evolution Guided by Large Language Models for Heterogeneous Flexible Job-Shop Scheduling with Cooperative Units

Hongquan Qu, Xiaomeng Tong, Shiliang Shao, Maolin Cai, Yan Shi, Yushan Ma, Fangwei Ning



PII: S1000-9345(26)00169-0

DOI: <https://doi.org/10.1016/j.cjme.2026.100374>

Reference: CJME100374

To appear in: *Chinese Journal of Mechanical Engineering*

Received date: 2 November 2025

Revised date: 17 July 2026

Accepted date: 22 July 2026

Please cite this article as: Hongquan Qu, Xiaomeng Tong, Shiliang Shao, Maolin Cai, Yan Shi, Yushan Ma and Fangwei Ning, Collaborative Evolution Guided by Large Language Models for Heterogeneous Flexible Job-Shop Scheduling with Cooperative Units, *Chinese Journal of Mechanical Engineering*, (2026) doi:<https://doi.org/10.1016/j.cjme.2026.100374>

This is a PDF of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability. This version will undergo additional copyediting, typesetting and review before it is published in its final form. As such, this version is no longer the Accepted Manuscript, but it is not yet the definitive Version of Record; we are providing this early version to give early visibility of the article. Please note that Elsevier's sharing policy for the Published Journal Article applies to this version, see: <https://www.elsevier.com/about/policies-and-standards/sharing#4-published-journal-article>. Please also note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Collaborative Evolution Guided by Large Language Models for Heterogeneous Flexible Job-Shop Scheduling with Cooperative Units

Hongquan Qu^a, Xiaomeng Tong^{a,b}, Shiliang Shao^a, Maolin Cai^a, Yan Shi^a, Yushan Ma^c, Fangwei Ning^{a,*}

^a School of Automation Science and Electrical Engineering, Beihang University, Beijing, 100191, China

^b State Key Laboratory of Precision Manufacturing for Extreme Service Performance, Central South University, Changsha, 410083, China

^c Liupanshan Laboratory, Yinchuan, 750021, China

Abstract

With the advancement of industrial intelligence, modern workshops increasingly rely on multiple intelligent collaborative units to jointly accomplish manufacturing tasks. To model this emerging production paradigm, this paper formulates the heterogeneous flexible job shop scheduling problem with collaborative units (HFJSCU) through a dedicated mathematical framework. To solve HFJSCU, we develop a novel memetic evolutionary framework driven by collaborative large language models (LLMs). The proposed framework operates in three stages. First, a preliminary memetic algorithm tailored for complex scheduling problems is automatically generated via multimodal prompting, without relying on predefined algorithmic templates. Second, the generated algorithm is decomposed into functional modules, which are iteratively refined through cooperation among multiple LLM agents, each responsible for specific components. Third, high-performing modules are selected and integrated to construct the final memetic algorithm. The proposed framework is validated on instances of different scales and types and compared with state-of-the-art LLM-assisted memetic algorithms. The experimental results demonstrate that the proposed framework outperforms the competing methods on 90% of the benchmark instances and exhibits significant superiority on the HFJSCU instances.

Keywords

Large language model, Memetic computing, Automated evolution framework, Flexible job-shop scheduling, Collaborative design algorithm

1 Introduction

Amid the accelerating wave of industrialization and technological innovation, manufacturing systems are confronted with increasingly complex and diversified production requirements. To respond to these challenges, enterprises are rapidly advancing toward intelligent and digital manufacturing paradigms. Within this transformation, flexible job shop scheduling (FJS) has emerged as a crucial technique owing to its capability to allocate operations among alternative machines adaptively [1]. Nevertheless, the classical FJS framework presumes homogeneous machine capabilities. In practical environments, newly introduced equipment often exhibits substantial differences in functionality and performance, especially when enterprises expand capacity to satisfy diverse order demands. To better capture such realities, heterogeneous flexible job shop scheduling (HFJS) has been developed [2]. Under HFJS settings, each operation can only be processed by a limited subset of eligible machines, and the corresponding processing durations differ across machines. Meanwhile, the widespread deployment of intelligent robots and digital management systems is fundamentally reshaping traditional production modes [3]. In conventional workshops, human workers are required to supervise and debug machines during operation execution, which tightly couples processing time with labor occupation time. In contrast, intelligent workshops enable autonomous adjustment and real-time monitoring, allowing robots to leave after completing preparatory tasks, thereby separating processing duration from setup or preparation time. Moreover, multi-robot collaboration in preparatory activities has become a representative feature of modern intelligent workshops. For instance, jobs may be retrieved from automated storage, transported to transfer stations, and then delivered to machines through coordinated robotic operations [4]. Motivated by these evolving occupation patterns and collaborative scheduling mechanisms, this study extracts the essential operational characteristics of intelligent workshops and formulates the heterogeneous flexible job shop scheduling problem with collaborative units (HFJSCU).

HFJSCU not only retains the combinatorial explosion characteristic of HFJS but also belongs to the class of NP-hard

*Corresponding author at School of Automation Science and Electrical Engineering, Beihang University, Beijing, 100191, China. E-mail address: nfangwei@buaa.edu.cn

Peer review under the responsibility of Editorial Board of Chinese Journal of Mechanical Engineering.

problems [5]. The introduction of decoupled processing and preparation stages, together with multiple collaborative units, further aggravates the computational burden. The intrinsic difficulty of HFJSCU can be summarized in three main aspects. 1) Decision Interdependence. In addition to operation sequencing and machine assignment required in FJS [6], HFJSCU requires determining the responsible collaborative unit for each operation's preparation and arranging the execution order of collaborative units, leading to tightly intertwined decisions. 2) Heterogeneity Across Multiple Dimensions. Variations exist not only in machine processing durations but also in the preparation times incurred by different collaborative units for distinct operations, introducing layered heterogeneity into the scheduling process. 3) Concurrent Coordination. Owing to the separation between preparation and processing stages, collaborative units may initiate preparation tasks on other machines while processing is ongoing. Effectively exploiting this concurrency is essential to prevent idle machines or collaborative units. These characteristics result in extensive decision alternatives and intricate temporal dependencies, giving rise to an enormous search space. As the scale of jobs, operations, machines, and collaborative units expands, the number of feasible schedules increases exponentially [7]. Consequently, solving HFJSCU necessitates decomposing the problem into interconnected sub-decisions, including operation sequencing, machine assignment, and collaborative unit scheduling, and coordinating their joint optimization to enhance production efficiency and resource utilization in intelligent workshops.

In general, solution strategies developed for HFJS can also be extended to HFJSCU [8]. Given its practical importance in manufacturing systems, this class of problems has long been a focus of sustained research interest. Existing approaches can be broadly categorized into four groups: exact algorithms, heuristic strategies, reinforcement learning methods, and metaheuristic techniques. Exact approaches, such as branch-and-bound [9], rely on rigorous mathematical formulations to obtain optimal schedules. Nevertheless, their applicability is often restricted to small or medium scale instances due to the exponential growth of computational complexity. Heuristic techniques, including dispatching rules and local search mechanisms [10, 11], are capable of producing satisfactory solutions within limited computational time. However, their restricted exploration range makes them susceptible to premature convergence and local optima. Reinforcement learning based methods, represented by deep Q-networks and proximal policy optimization [12, 13], employ end-to-end learning frameworks to generate scheduling decisions efficiently. Despite this advantage, their underlying search behavior remains closely related to heuristic paradigms, resulting in comparable solution quality and limited robustness across diverse problem settings. Metaheuristic algorithms, particularly evolutionary algorithms (EA) [14, 15], enhance population diversity through global exploration mechanisms. Yet, for large-scale combinatorial optimization problems, further improvements in solution quality are still desirable. Memetic algorithms (MAs), as an important subclass of metaheuristics, combine evolutionary global search with problem-specific local refinement. This hybrid structure has demonstrated strong effectiveness in combinatorial optimization and has therefore become a dominant approach for shop scheduling [16]. However, the performance of MAs depends heavily on the coordinated design of multiple functional components. Such designs are typically guided by expert knowledge and manual parameter tuning, which constrain their level of autonomy and limit generalization capability.

Recently, optimization approaches driven by large language models (LLMs) have emerged as a rapidly growing research direction [17]. Although LLMs and evolutionary algorithms (EAs) originated from distinct research domains, notable conceptual parallels between them provide the theoretical basis for LLM-enhanced EAs [18]. Leveraging the semantic reasoning and code synthesis capabilities of LLMs, therefore, offers new possibilities for the automated construction of memetic algorithms aimed at solving complex shop scheduling problems. From the perspective of prompting strategies, most existing LLM-based algorithm design methods rely on textual prompts [19] or in-context learning demonstrations [20]. While these approaches are effective for relatively simple tasks, their performance degrades when addressing highly complex optimization problems. With the advancement of multimodal LLMs equipped with visual understanding capabilities, visual prompting has attracted increasing attention. Shtedritski et al. [21] showed that annotated visual regions can guide LLMs to concentrate on relevant content. In addition, multimodal prompting has been successfully applied in image classification [22], image generation [23], and model training [24], further demonstrating its potential to enhance LLM performance. Regarding algorithm design paradigms, Google DeepMind introduced the FunSearch framework [25], which integrates LLM reasoning with evolutionary principles to automatically construct heuristic algorithms, establishing a new direction for algorithm discovery. Liu et al. [26] proposed the Evolution of Heuristics (EoH) framework, where LLMs iteratively generate and refine heuristic strategies within a closed-loop evolutionary process without human intervention. Ye et al. [27] incorporated LLM-based feedback to improve heuristic

exploration efficiency. Pluhacek et al. [28] combined a self-organizing migratory algorithm with LLM-generated components for iterative refinement in the absence of feedback. Stein et al. [29] developed LLaMEA by emulating evolutionary procedures to construct metaheuristic algorithms for continuous optimization, and further integrated hyperparameter optimization to reduce computational cost. More recently, Li et al. [30] proposed the LLM-assisted memetic algorithm (LLMMA), in which a predefined EA is provided to an LLM that generates effective local search operators to enhance overall solution quality. The overall frameworks of these representative studies are illustrated in Fig. 1.

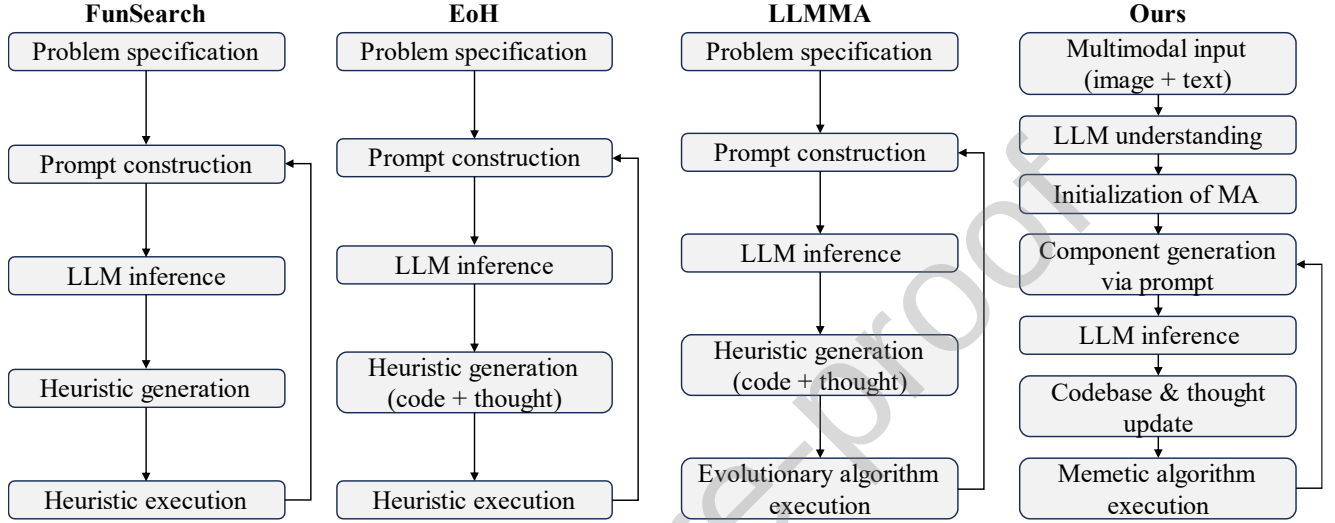


Fig. 1 Workflow diagrams of different frameworks

Although some of the aforementioned studies involve job shop scheduling, their experimental settings are generally limited to simplified variants. This limitation stems from the fact that automatically generated heuristic methods require precise and detailed problem formulations when dealing with highly complex tasks [26]. When extending LLM-driven memetic algorithm design to complex problems such as HFJSCU, several critical challenges emerge. 1) Excessive Prompt Length. As problem complexity increases, the corresponding prompts become longer and more intricate. Long-context inputs may trigger the "lost in the middle" phenomenon [31], where essential information is diluted within extensive instructions. Consequently, overly concise prompts fail to produce complete memetic algorithms for HFJSCU, whereas overly detailed prompts degrade model focus. 2) Algorithm-Oriented Prompt Bias. To facilitate model comprehension, existing automated metaheuristic design frameworks frequently incorporate well-known algorithms into the prompts [32]. While helpful for initialization, this practice constrains the search space and implicitly steers the LLM toward reproducing established algorithmic patterns rather than exploring genuinely novel structures. 3) Reliance on a Single LLM: Most current frameworks construct closed-loop systems around a single LLM instance. After repeated generation cycles, this setup increases the likelihood of reproducing similar algorithmic structures, thereby reducing innovation and inducing design stagnation [33]. 4) Elimination of Imperfect Outputs. LLM-generated algorithms may contain creative structural ideas even if their implementations are not directly executable. Discarding such outputs entirely diminishes population diversity and weakens exploratory potential.

In this study, we develop the Multi-LLM Collaborative Evolution of Memetic Algorithms (MLCEMA) framework. By exploiting the multimodal capabilities of LLMs, particularly visual prompting, the framework enhances model perception and reasoning for complex scheduling scenarios. Through coordinated optimization among multiple LLM agents, MLCEMA enables the automatic construction of memetic algorithms tailored to HFJSCU. The optimization objective considered in this work is the minimization of makespan. The main contributions are summarized as follows.

1) Problem Formulation: We introduce the HFJSCU problem, which models interaction constraints among collaborative units, dependencies between collaborative units and operations, and their heterogeneous characteristics. Compared with classical FJS and resource-continuous occupation settings, HFJSCU provides a more realistic representation of intelligent

workshop environments.

2) Methodology: We design a multi-role, LLM-driven collaborative evolutionary framework for memetic algorithm generation. To the best of our knowledge, this is the first attempt to employ LLMs to autonomously generate and iteratively refine a complete memetic algorithm capable of matching or surpassing existing approaches. The framework adopts an LLM-leader mechanism combined with a divide-and-conquer strategy to integrate algorithmic components produced by different LLM agents. Initially, visual prompting assists the LLM-leader in constructing a foundational algorithm structure, thereby reducing manual development effort. Subsequently, the overall strategy is decomposed into independent modules, which are separately optimized by LLM collaborators. The updated modules are evaluated, and high-performing components are stored in a strategy repository. Through iterative sampling, evaluation feedback, and prompt refinement guided by the LLM-leader, collaborators are cyclically invoked to achieve progressive improvement. Ultimately, elite modules are assembled to form the final memetic algorithm, termed the Multi-LLM Collaborative Memetic Algorithm (LCMA). This architecture alleviates the cognitive burden on a single agent and enhances combinatorial innovation in automated algorithm design.

3) Algorithm Application: A problem-specific decomposition mechanism for HFJSCU is proposed, where multi-level encoded visual representations assist the LLM-leader in structural understanding. The MLCEMA framework is evaluated on benchmark HFJS datasets as well as HFJSCU instances of various scales. Experimental results indicate that MLCEMA demonstrates strong generalization ability, and the generated LCMA achieves superior solution quality compared with current state-of-the-art methods.

The remainder of this paper is organized as follows. Section 2 presents the formal definition of HFJSCU. Section 3 details the MLCEMA framework and its functional components. Section 4 reports the experimental setup and empirical results. Section 5 concludes the paper and outlines future research directions.

2 Problem Description and Mathematical Model

2.1 Problem Description

An HFJSCU instance can be characterized as follows.

Jobs and Operations: Consider a set of jobs J_i ($i = 1, \dots, N$). Each job J_i contains K_i sequential operations denoted by $O_{i,j}$ ($j = 1, \dots, K_i$). Precedence constraints exist within each job, such that operation $O_{i,j+1}$ cannot start until operation $O_{i,j}$ has finished.

Heterogeneous Machines: Let M_k ($k = 1, \dots, M$) represent the set of available machines. Each operation $O_{i,j}$ can only be processed on a subset of machines M_k , which is a subset of all machines. Moreover, the processing time $T_{i,j,k}$ depends on the selected machine M_k , reflecting machine heterogeneity.

Collaborative Units: Let CU_l ($l = 1, \dots, C$) denote the collaborative units responsible for preparatory activities prior to processing. The preparation time $T_{i,j,l,prep}$ varies across different units, capturing differences in their capabilities.

Separation of Preparation and Processing: During the execution of operation $O_{i,j}$ on machine M_k , a collaborative unit CU_l may simultaneously prepare subsequent operations, such as $O_{i,j+1}$. Therefore, the starting time $S_{i,j}$ of operation $O_{i,j}$ is determined by three factors: the completion of its predecessor $O_{i,j-1}$, the availability of the selected machine M_k , and the completion of the required preparatory tasks performed by the assigned collaborative unit CU_l .

To simplify the formulation, the following assumptions are adopted:

- (1) At time $t = 0$, all machines, collaborative units, and jobs are available.
- (2) Collaborative units function as auxiliary resources supporting both operations and machines.
- (3) Each operation requires two collaborative units to perform preparation simultaneously.
- (4) The working duration of collaborative units is independent of job precedence relationships.
- (5) Operations are processed without interruption.
- (6) Machine failures and stochastic disturbances are not considered.

2.1 Mathematical Model and Illustrative Instance

This subsection presents a brief mathematical formulation of the HFJSCU problem, followed by a representative instance to illustrate its key characteristics. For clarity and conciseness, the detailed definitions of indices, sets, parameters, decision variables, and all constraint formulations are provided in the supplementary file.

HFJSCU extends the classical heterogeneous flexible job shop scheduling framework by incorporating preparation tasks performed by collaborative units and decoupling processing time from preparation time. The model simultaneously determines operation sequencing, machine assignment, collaborative unit allocation, and their respective scheduling relationships under heterogeneous constraints.

The objective of HFJSCU is to minimize the makespan, defined as the maximum completion time among all jobs:

$$F = \min C_{\max}. \quad (1)$$

The complete mixed-integer programming formulation, including all variable definitions and constraint descriptions, is presented in the supplementary file.

To further illustrate the characteristics of HFJSCU, a simplified instance is constructed, and its corresponding schedule is shown in Fig. 2. In this example, each operation requires the cooperation of two collaborative units. The production task consists of two jobs, each comprising two operations. The workshop includes two heterogeneous machines and three heterogeneous collaborative units.

According to the generated schedule, the preparation sequence of collaborative units for operation $O_{2,1}$ is arranged as CU_3 followed by CU_1 , which prevents temporal overlap with the collaborative units assigned to $O_{1,1}$. This example demonstrates that, under HFJSCU, decision-making must address not only the selection of suitable heterogeneous collaborative units but also the coordination of their execution order to avoid resource conflicts.

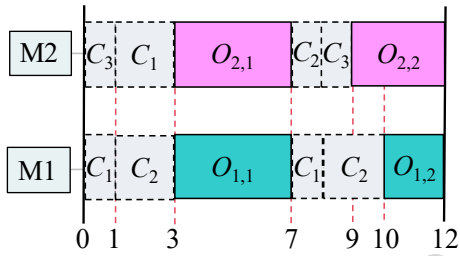


Fig. 2 Gantt chart of the HFJSCU scheduling results

3 LLM Collaborative Evolution of Memetic Algorithms

3.1 Framework of MLCEMA

As shown in Fig. 3, MLCEMA operates through an iterative mechanism driven by multimodal prompting and LLM collaboration. Multimodal prompting is introduced in place of conventional problem-specification modules, thereby reducing the technical threshold for applying metaheuristic methods. The LLM-leader interprets these multimodal inputs and constructs an initial memetic algorithm (I-MA). Subsequently, the I-MA is decomposed into functional modules, and paired code-prompt combinations are delivered to LLM-collaborators for refinement. After evaluation, the generated modules are archived in a strategy repository and treated as parent individuals for subsequent iterations. This procedure is repeated, while the temperature of the LLM-collaborators is adaptively regulated to generate a diversified initial population. Candidate individuals are then sampled from this population, and the LLM-leader inspects erroneous implementations and augments the prompts with corrective instructions. The revised prompts, together with the selected code modules, are resubmitted to the corresponding LLM-collaborators for further evolution. Through iterative evolution, high-quality strategy modules are progressively generated. Ultimately, the best-performing modules are extracted and assembled into the I-MA framework, forming the final algorithm for solving the HFJSCU.

3.2 Process Description

Multimodal Prompt (Initialization): The evolutionary trajectory of MLCEMA is fundamentally determined by its initial prompt configuration, which defines the semantic and structural constraints under which the LLM-leader operates. Early exploratory experiments revealed that text-only prompting frequently failed to produce effective memetic algorithms, even for relatively simple FJS instances [34]. This observation suggests that purely linguistic representations lack the expressive capacity to encode the intricate structural and temporal dependencies inherent in workshop scheduling problems. To enhance representational fidelity, a multimodal prompting mechanism is introduced. **Visual Prompts:** An encoded structural image conveys the problem class and its combinatorial characteristics, whereas a simplified Gantt chart encodes temporal dependency patterns within the scheduling solution space. **Text Prompts:** The textual component specifies role definitions, instance descriptions, output constraints, and explanatory annotations of the visual encoding. As illustrated in Fig. 4 and Prompt Template A, a concise description of the image encoding protocol is embedded to align visual abstraction with symbolic reasoning. This structured multimodal representation enables the LLM-leader to systematically decompose HFJSCU into solvable subproblems during the generation of the initial memetic algorithm (I-MA).

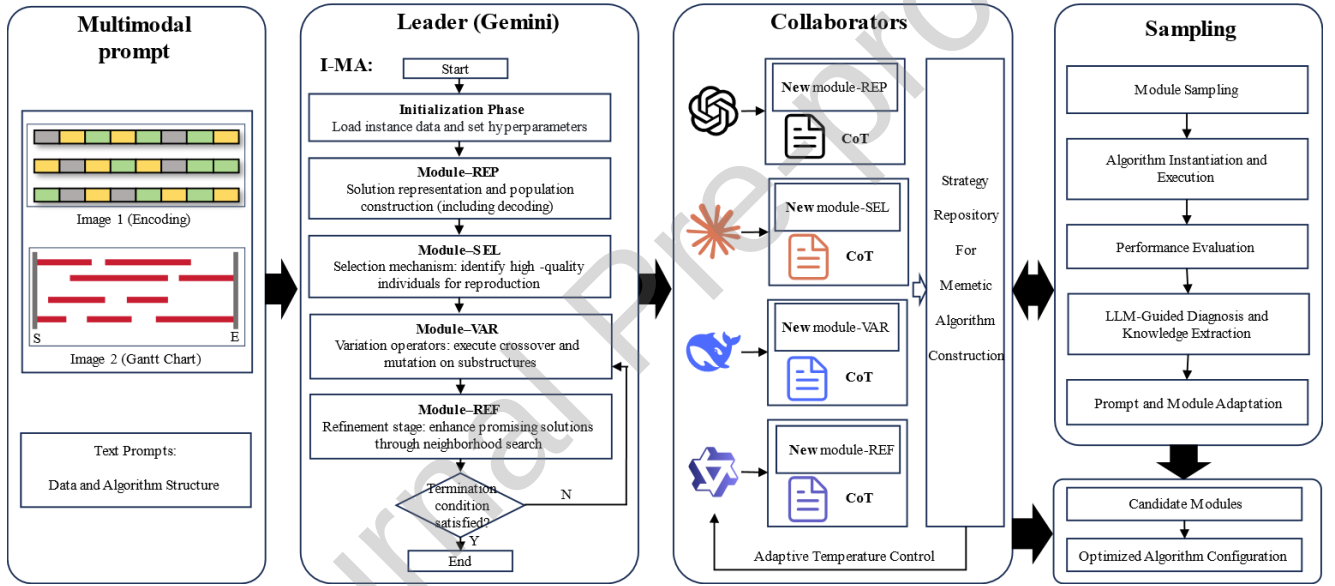
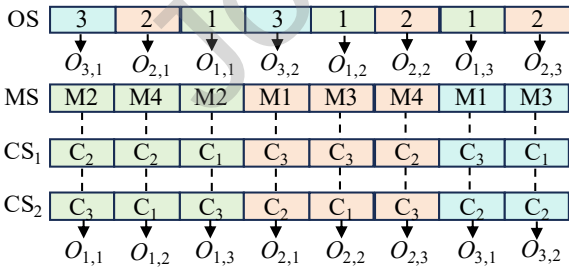
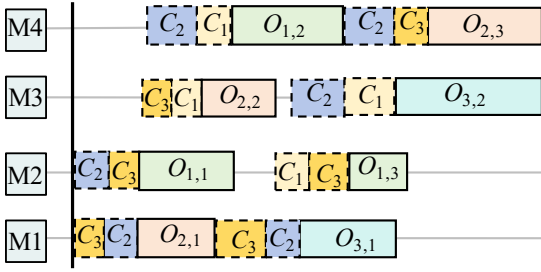


Fig. 3 LLM-Driven Collaborative Evolution Framework



(a)



(b)

Fig. 4 Illustration of the visual prompting components: (a) Multi-layer encoding, (b) Resulting scheduling layout

Prompt Template A	
●	Act as an expert in memetic algorithm design.
●	Generate executable Python code for HFJSCU.
●	Image 1: encoding scheme; Image 2: scheduling structure.
●	Provide instance configuration and data format.
●	Follow the specified module naming rules.
●	Output constraints: no explanation; report the best objective and schedule.

LLM Role: Distinct LLMs possess heterogeneous prior knowledge distributions due to differences in training corpora and architectural bias [35]. Such diversity naturally induces variation in reasoning pathways and search behaviors[36]. Multi-LLM collaborative paradigms have demonstrated enhanced robustness and performance across domains including medical reasoning and large-scale system engineering [37]. Building upon this premise, MLCEMA formalizes a role-differentiated collaborative structure consisting of an LLM-leader and multiple LLM-collaborators. The LLM-leader performs global coordination and structural governance, including: (1) interpreting multimodal inputs, (2) synthesizing the foundational architecture of the memetic algorithm, and (3) diagnosing structural or logical inconsistencies in collaborator-generated code. This role requires integrated visual reasoning and high-level program synthesis capabilities beyond elementary visual extraction. Error diagnosis is conducted with reference to a baseline module provided in the supplementary file. The LLM-collaborators operate at the strategy-module level, analogous to evolutionary operators in classical MAs. Their contribution is dynamically regulated through a performance-driven selection mechanism:

$$P_i^{(t+1)} = \frac{P_i^{(t)} \cdot \Delta_i^{(t)}}{\sum_{j=1}^N P_j^{(t)} \cdot \Delta_j^{(t)}}, \quad (2)$$

where $P_i^{(t)}$ denotes the participation probability of collaborator i at iteration t ; N is the number of collaborators, and $\Delta_i^{(t)}$ quantifies its evolutionary contribution. This formulation induces a replicator-dynamics-like update mechanism, promoting high-performing strategy modules while maintaining diversity. To prevent premature elimination, the normalized probability is lower bounded as $p_i = \max(p_i', \epsilon)$, ensuring sustained exploration capacity.

Module Extraction and Merging: The initial algorithmic structure synthesized by the LLM-leader is decomposed into core functional components, including encoding and initialization, environmental selection, crossover and mutation, and local search. Owing to potential hallucinations and structural redundancies introduced by LLM generation, additional auxiliary functions may also appear in the produced code. To systematically isolate strategy-related segments, abstract syntax tree (AST) analysis and regular expression matching are employed. AST parsing facilitates the identification of undefined or auxiliary functions [38], enabling structural separation of algorithmic components. During module integration, namespace consistency must be maintained to ensure that variables and functions remain globally coherent. The merging sequence follows a hierarchical order: strategy modules, auxiliary functional modules, global variables, and finally the program entry point. To fairly assess the contribution of individual strategy modules, each module is embedded and executed independently within the framework.

Exception Handling: To enhance robustness during automated evolution, runtime anomalies are monitored and categorized. For execution failures resulting in abnormal termination, the captured error message is transformed into corrective prompt information. In cases of excessive computational latency, a predefined time constraint is enforced to avoid stagnation, and additional guidance related to computational efficiency is introduced. When the program completes execution but produces infeasible or abnormal outputs, constraint-related feedback is appended to the prompt. If such issues arise during the initialization phase, the refined prompts are integrated into Prompt Template A, and the updated multimodal configuration is resubmitted to the LLM-leader to regenerate the I-MA.

Module Prompts (Evolution): The production of new candidate strategies is driven by prompt-level operators. Following the evolutionary prompting idea in Ref. [26], five prompt operators are constructed, denoted as [D, V, M, S, R], to induce controlled variation in strategy modules. Specifically, D (Diversify) produces a structurally distinct strategy independent of the current module; V (Vary) introduces alternative designs while inheriting key optimization principles; M (Modify) adjusts algorithmic parameters within predefined feasible ranges; S (Simplify) reduces structural complexity to alleviate computational burden; and R (Rectify) performs targeted correction of faulty modules. The detailed prompt templates are provided in Prompt Template B in the supplementary file. The invocation frequency of D and V is controlled by preset probabilities. The M operator is reserved for parameter tuning of high-performing LCMAs, whereas S is triggered by excessive runtime behavior, and R is activated whenever structural or logical anomalies are detected.

The strategy modules generated under these operators incorporate a reasoning trace in the form of a chain-of-thought (CoT). The CoT mechanism enhances the reliability of code synthesis and mitigates hallucination risks [39]. For each evolved module, the LLM is required to explicitly articulate its reasoning process prior to final code generation. This intermediate reasoning stage improves instruction adherence and strengthens the alignment between evolutionary operator intent and the resulting implementation.

Prompt Template B	
●	Act as a memetic algorithm specialist.
●	The following module <i>{module_name}</i> (rationale in triple quotes) is designed for HFJSCU. <i>{target_code}</i>
●	Runtime error information: <i>{error_information}</i> Detected Issue:
●	Compare with the reference module and identify structural inconsistencies. <i>{baseline_module}</i> Analysis Task:
●	Provide a concise diagnosis and corrective strategy for prompt revision. Output Constraints:
1.	Keep the original code unchanged.
2.	No explanatory text.
3.	Output only the diagnosis and correction in triple quotes: """ Error cause and modification suggestions: xxx """

Exception Handling: Strategy module execution may still produce runtime inconsistencies, similar to those observed during I-MA evaluation. In most cases, the error manifests within the decoding stage rather than the strategy module itself. To isolate the root cause, the newly generated module is compared against a baseline reference module. The LLM-leader analyzes structural discrepancies and formulates a corrective prompt. When the affected module is selected for further evolution, the R operator integrates the correction instructions into the next generation prompt, thereby improving robustness and preventing repeated structural faults. The prompt template employed for LLM-leader-based exception handling is given below.

Evaluation and Sampling: The assessment of newly generated strategy modules constitutes a core control mechanism

within the LLM-driven evolutionary loop, as it directly influences subsequent prompt adaptation and module survival. The evaluation procedure includes the following components: verifying the functional validity of the module, detecting and reporting constraint violations under feasible execution, quantifying makespan improvement, measuring execution-time variation, and updating the temperature parameter. Runtime anomalies are handled according to the previously defined exception mechanism. In addition, scheduling outcomes may appear syntactically correct yet fall below the theoretical lower bound. To detect such cases, Prompt Template A requires the LLM to output detailed schedule information, enabling explicit constraint verification. The corresponding constraint-checking algorithm is provided in Algorithm 2 in the supplementary file. Makespan gains and execution-time differences are computed from module-level performance logs. When the execution-time increment exceeds a predefined threshold, the S operator is triggered to control computational cost. Temperature regulation follows a strategy analogous to adaptive parameter control in evolutionary algorithms: higher temperatures are applied in early stages to encourage exploration and diversity, while lower temperatures are adopted in later stages to promote stability and refinement. Validated modules are then archived into their respective knowledge bases to support subsequent sampling.

Sampling: Module selection is governed by an efficiency-based probabilistic mechanism implemented via a roulette wheel sampling. The efficiency factor is defined as:

$$EF = \frac{1}{\Delta T} \times \frac{C_{ref}}{C_{alg}}, \quad (3)$$

where ΔT denotes the execution-time variation introduced by the strategy module, C_{ref} is the makespan prior to applying the module, and C_{alg} is the resulting makespan after integration. Modules producing incorrect or infeasible solutions are penalized by adjusting ΔT and C_{alg} such that their corresponding EF remains inferior to that of valid competitors, thereby reducing their selection probability in subsequent iterations.

3.3 Synthesis of the LCMA

The LCMA is constructed by integrating high-quality strategy modules evolved within the MLCEMA framework, as depicted in Fig. 5. In contrast to conventional approaches that rely on a predefined algorithm as the starting point, the structural backbone of LCMA originates from the I-MA synthesized by the LLM-leader. The I-MA is first executed on the HFJSCU problem to generate an initial solution, which serves as a performance reference for subsequent LLM-driven refinement. Preset strategy modules are retained only as diagnostic references to assist the LLM-leader in analyzing faulty outputs. The MLCEMA framework then iteratively produces diverse candidate modules. To avoid constraining the search process within the conceptual boundaries of existing memetic algorithms, no complete preset algorithm is supplied as input; instead, a modular design paradigm is adopted to preserve structural flexibility and exploratory capacity. Ultimately, the most competitive strategy modules are selected and substituted into the corresponding components of the I-MA, yielding the finalized LCMA.

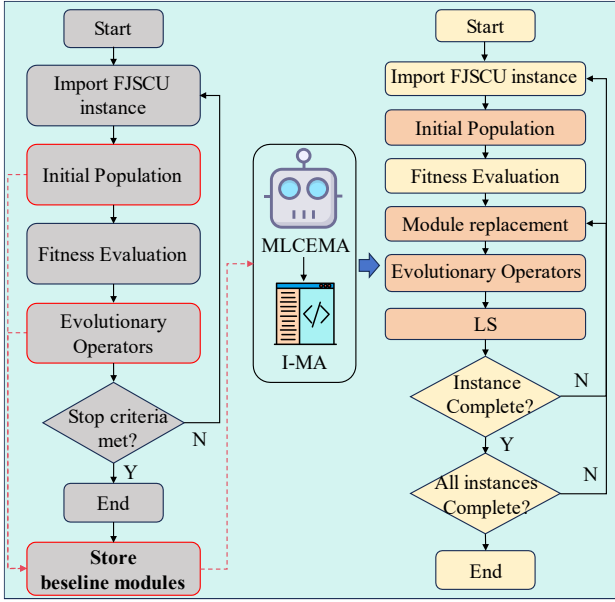


Fig. 5 Workflow of LCMA construction

4 Experimental Comparison and Analysis

This section begins by assessing the validity and reliability of the multimodal prompting mechanism, followed by a description of the adopted benchmark datasets. Subsequently, hyperparameter calibration, ablation analysis, and performance comparisons against competing algorithms are reported. All experiments were conducted on a Microsoft Windows 11 platform equipped with an Intel Core i9 (3.9 GHz) processor, 64 GB RAM, and an NVIDIA GeForce RTX 4080 GPU. The implementation was developed in Python 3.10. Large language models were invoked via their official SDK interfaces using non-streaming mode, ensuring that complete responses were returned in single-request executions.

4.1 Large Language Models

At present, widely adopted large language models include ChatGPT, Claude, Gemini, Qwen, and DeepSeek. The role of the LLM-leader necessitates multimodal reasoning capability. As DeepSeek, Qwen3-plus, and Qwen3-max provide only text-based interfaces, and Qwen-QVQ-Max supports streaming outputs exclusively, these models were deemed unsuitable for this function. Consequently, ChatGPT-o3, Claude-opus-4, Gemini-2.5-pro, and Qwen-VL-Max were evaluated by supplying them with both the encoded representation and the corresponding scheduling visualization of an HFJSCU instance. The temperature parameter T was configured at $\{0, 0.5, 1\}$, with five independent trials conducted for each setting. To rigorously assess visual reasoning capacity, an intentional inconsistency was introduced between the encoded structure and the scheduling result, without any explanatory cues. Specifically, the encoded image indicated four machines, whereas the scheduling chart displayed only three. Model outputs were manually examined following the evaluation protocol in Ref. [40], and response completeness is summarized in Table 1. The findings indicate that all four models correctly identified the problem as belonging to the scheduling domain without textual hints. Except for Claude-opus-4, the remaining models further recognized it as an extension of FJS involving multiple subcomponents. However, only Gemini-2.5-pro successfully detected the structural inconsistency between the encoding and the scheduling representation.

Table 1 Comparison of multimodal understanding capabilities across LLMs

Capability category	Specific task	Claude	GPT	Qwen	Gemini
Domain understanding	Scheduling domain recognition	✓	✓	✓	✓
	Substructure	✓	✓	✓	✓

level reasoning	decomposition			
	Operation dependency interpretation	×	✓	✓
Machine allocation reasoning	Machine allocation reasoning	×	✓	✓
	Collaborative resource understanding	×	✓	✓
Schedule visualization	Constraint awareness	✓	✓	✓
Cross-modal consistency	Structural inconsistency detection	×	×	×

4.2 Dataset and Performance Metrics

Dataset. The HFJS benchmark instances are adopted from the widely used datasets in Refs. [41–43]. For the proposed HFJSCU problem, a dedicated dataset is constructed based on the MK benchmark series, comprising 30 instances of different scales [41]. The parameter configurations are summarized in Table 2. For large-scale cases ($N \geq 30, M \geq 20, K_i \geq 160$), the candidate machine set size $M_{i,j}$ is randomly sampled from Refs. [2,30], while ensuring that it remains smaller than the total number of available machines in each instance.

Performance Metrics. To quantify the capability of LLM-generated memetic algorithms, the overall evaluation score (OES) is defined as:

$$OES_i = \frac{1}{X} \sum_{j=1}^X \left(\frac{B_i}{\delta_{i,l} R_{i,l} + (1 - \delta_{i,l}) \gamma} \right), \quad (4)$$

where l denotes the run index, $\delta_{i,l}$ is an indicator variable equal to 1 if execution is successful and 0 otherwise, and γ is a penalty coefficient. B_i corresponds to the baseline objective value for instance i , and $R_{i,l}$ represents the outcome of the l -th run on that instance. The penalty term $(1 - \delta_{i,l}) \gamma$ accounts for infeasible runs or timeouts. Larger OES values imply superior algorithmic performance. Based on the rationale of OES, the relative improvement (RI) metric is adopted to evaluate LCMA, defined as:

$$RI = \frac{M_{\text{base}} - M(A_0)}{M_{\text{base}}}, \quad (5)$$

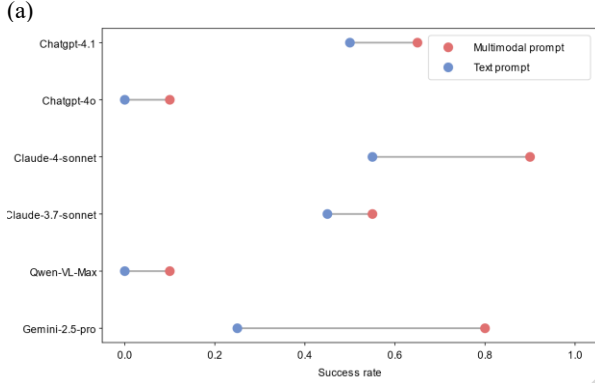
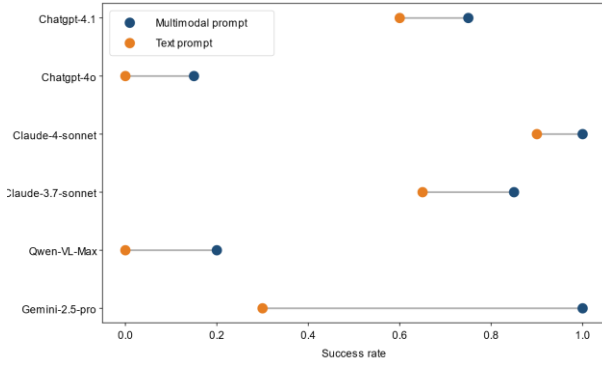
where $M_{\text{base}} = \min_{j \in \{1, \dots, N\}} \{M(A_j)\}$.

4.3 Multimodal Prompting Performance Evaluation

This subsection examines the reliability of LLM-generated code on the HFJS and HFJSCU benchmarks. While Section 4.1 focuses on evaluating problem comprehension, the objective here is to assess the capability of LLMs to produce executable solutions for workshop scheduling tasks. The evaluated models include Gemini-2.5-pro, Qwen-VL-Max, Claude-3.7-sonnet, Claude-4-sonnet, ChatGPT-4o, and ChatGPT-4.1. The experiments first verify the correctness of code generated for the classical HFJS problem and then extend the evaluation to the more complex HFJSCU scenario. For both problems, a controlled comparison is conducted: one group employs the proposed multimodal (image + text) prompts, while the other relies solely on textual prompts. Each configuration is executed 10 times under $T = 0$ with a 300-second timeout to compute the success rate.

Fig. 6 shows that multimodal prompting consistently achieves higher success rates than text-only prompting across all tested models. In particular, Claude-4-sonnet and Gemini-2.5-pro exhibit strong performance on HFJSCU instances. For Gemini-2.5-pro, multimodal prompting increases the success rate by 70% on HFJS and 60% on HFJSCU compared with text-only prompting. Given its superior reasoning on complex scheduling tasks and robust code synthesis ability, Gemini-2.5-pro is selected as the LLM-leader. Considering computational efficiency and operational cost, the LLM-collaborator set includes Gemini-2.5-flash, ChatGPT-4o, Claude-sonnet-4.0, Qwen-max, and DeepSeek-v3. These findings further suggest that textual information alone is often insufficient for conveying the structural semantics of complex scheduling

problems, whereas visual cues enhance algorithm generation performance.



(b)

Fig. 6 Success rates of LLM-generated scheduling algorithms under different prompting strategies: (a) HFJS benchmark, (b) HFJSCU benchmark

4.4 Hyperparameter Configuration

The configuration of hyperparameters plays a decisive role in the optimization performance of LCMA. The MLCEMA framework includes four key tunable parameters: (1) population size $ps = \{40, 60, 80\}$; (2) iteration number $iter = \{10, 15, 20\}$; (3) selection ratios of evolutionary operators $[D, V] = \{[0.7, 0.3], [0.6, 0.4], [0.5, 0.5]\}$; and (4) maximum temperature $T_{max} = \{0.5, 0.7, 0.9\}$. To systematically analyze their effects, a Taguchi orthogonal design with four factors at three levels was adopted, resulting in nine experimental configurations, as illustrated in Table 2. To ensure fair comparison across parameter settings, all experiments were initialized with the same I-MA. If the framework were applied to the entire dataset for each configuration, the total number of LLM invocations would scale proportionally with the number of instances, experimental configurations, independent runs, and the combined cost of population initialization and iterative evolution. Even under the minimum setting, the number of LLM calls exceeds 3.24×10^4 per run, indicating substantial computational cost. Therefore, to balance evaluation rigor and resource expenditure, training was conducted only on the medium scale instance N30M20CU20 ($K_i = 161$). The procedure was repeated over five independent executions, each producing an LCMA through the selection and integration of the best-performing strategy modules. These resulting LCMA were subsequently evaluated on all instances, and ten runs were performed in total across the five executions to compute averaged results.

Table 2 Orthogonal experimental design with four factors and three levels

ID	ps	$iter$	$[D, V]$	T_{max}
1	60	10	[0.6, 0.4]	0.5
2	80	10	[0.5, 0.5]	0.7
3	80	15	[0.7, 0.3]	0.5

4	60	15	[0.5, 0.5]	0.9
5	80	20	[0.6, 0.4]	0.9
6	40	20	[0.5, 0.5]	0.5
7	60	20	[0.7, 0.3]	0.7
8	40	10	[0.7, 0.3]	0.9
9	40	15	[0.6, 0.4]	0.7

The main effects diagram of the orthogonal experiments is presented in Fig. 7. The slope magnitude reflects the relative impact of each factor, among which ps exhibits the most pronounced effect, as indicated by the steepest curve. The directional trends of the curves reveal the preferable levels for each parameter. Since the objective is to maximize OES , the preliminary optimal configuration appears to be $ps = 60$, $iter = 15$, $[D, V] = [0.5, 0.5]$, and $T_{max} = 0.5$. However, closer inspection shows that the superior OES achieved by the fifth group was primarily driven by a single exceptionally strong module generated in one run, which disproportionately enhanced the overall LCMA performance. A similar phenomenon has been reported in Ref. [25].

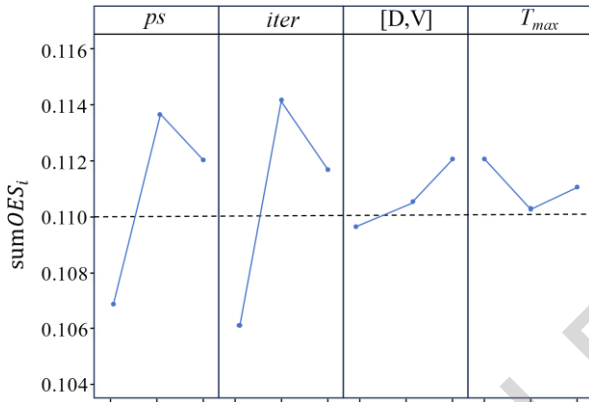


Fig. 7 Main effects of four hyperparameters at three levels

To mitigate the influence of such outlier effects, we further examined the entire population of strategy modules produced throughout the evolutionary process. Instead of relying on peak results, this analysis evaluates the collective contribution of all modules in terms of their improvement over the baseline I-MA, thereby enabling a more statistically robust parameter calibration. The detailed outcomes are illustrated in Fig. 8. Although the fifth configuration achieved the highest single-run performance, the last three configurations produced a larger number of high-quality modules and demonstrated superior average and median results. This observation suggests that a larger population enhances exploration capacity and stabilizes the evolutionary trajectory, favoring $ps = 80$. Consequently, the final parameter setting was determined as $ps = 80$, $iter = 15$, $[D, V] = [0.5, 0.5]$, and $T_{max} = 0.5$.

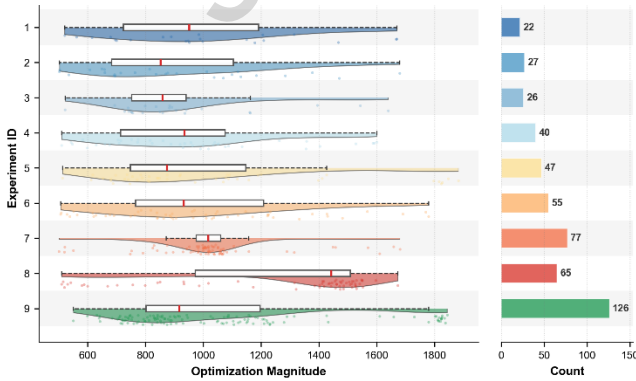


Fig. 8 Distribution of optimization magnitude and elite solution counts across experimental configurations

4.5 Ablation Study

To examine the effectiveness of the principal components within the automated memetic design paradigm, an ablation analysis was performed on the MLCEMA framework. The experimental configuration followed Section 4.4, with a computational budget of 600 seconds. Performance was assessed using Eq. (4), where the penalty coefficient was set to $\gamma = 1.5B_i$.

Beyond multimodal prompting, the essential mechanisms of MLCEMA encompass modular decomposition of strategies, multi-LLM cooperative evolution, specialized evolutionary operators, and the reuse of erroneous code for knowledge refinement. Accordingly, four controlled variants were constructed. (1) No Modularization: the framework directly generates a complete algorithm without strategy decomposition (Gemini-2.5-flash), denoted as MA1. (2) Single LLM: Strategies are modularized, but evolution is conducted by a single LLM (Gemini-2.5-flash), resulting in MA2. (3) Simplified Operators: Multi-LLM collaboration is preserved, yet the D, V, and M operators are replaced by a unified operator E with the prompt “Generate a better algorithm,” while S and R are retained for exception handling; this variant is labeled LCMA1. (4) Discarding Errors: faulty strategy code is excluded from the population. Since this modification may implicitly enlarge the effective population beyond 80, the population size was fixed at $ps = 80$ to maintain fairness, and only valid code together with its reasoning was admitted as parent individuals; this variant is termed LCMA2. For consistency, all variants evolved from the same I-MA. The evolving modules comprised initialization, environmental selection, crossover and mutation, and local search. Each configuration was executed five times on the N30M20CU20 instance; the best module from each run was retained, and the resulting LCMA2s were evaluated over ten independent runs to report averaged outcomes.

An analysis of the solution performance across different variants leads to several observations. Except for MA1, all other configurations, when evolved from the I-MA, achieved substantial improvements over the baseline, indicating that LLM-guided refinement can progressively enhance algorithm quality, whereas constructing an effective algorithm entirely from scratch remains challenging for complex tasks. During experimentation, MA2 and LCMA1 frequently encountered difficulties in obtaining feasible solutions within acceptable time limits on large-scale instances. This behavior suggests that evolution driven by a single LLM lacks sufficient global coordination, tending to overfit local structures of specific instances and exhibiting limited scalability. Likewise, reliance on a single prompt template may induce uncontrolled exploratory shifts, causing the search trajectory to become unstable and directionless. Consequently, the strategy modules generated in LCMA1 often exhibited excessive ineffective exploration and internal resource consumption.

Comprehensive training outcomes are reported in Table 3. The aggregated scores are $OES_{sum}(MA1) = 3.308$, $OES_{sum}(MA2) = 5.881$, $OES_{sum}(LCMA1) = 5.286$, $OES_{sum}(LCMA2) = 7.657$, and $OES_{sum}(LCMA) = 7.840$. These results quantify the contribution of individual components within the MLCEMA framework. The consistent superiority of all variants over MA1 confirms the benefit of strategy modularization. In particular, MA2 achieved optimal solutions on four instances and surpassed LCMA1 in most cases, whereas LCMA2 also reached optimality on four instances and generally exceeded MA2, indicating that multi-LLM collaboration provides additional performance gains. The fully integrated LCMA delivered optimal results on 22 instances, underscoring the importance of preserving and reusing erroneous strategy code within the evolutionary process.

A comparative assessment of component significance is summarized in Table 4, which presents pairwise dominance relations along with Elo rating scores. Originally developed for ranking chess players and now widely adopted for evaluating LLM systems [44], the Elo framework provides a quantitative measure of relative performance. The final LCMA surpasses MA1, MA2, LCMA1, and LCMA2 on 30, 26, 30, and 25 instances, respectively, further substantiating the necessity of integrating all core components within the collaborative evolutionary architecture.

Within the Elo framework, each variant begins with a base score of 1000 and is updated using a K -factor of 32. The magnitude of Elo reduction under ablation conditions (termed “Ablation-Drop”) reflects the relative importance of each component. The results indicate that strategy modularization constitutes the most indispensable element, as it establishes the structural foundation of MLCEMA. Evolutionary operators rank next in influence, functioning as the decision-driving mechanism that steers the search trajectory. Multi-LLM collaborative evolution further contributes by amplifying diversity and expanding the attainable performance ceiling. Finally, preserving erroneous strategies enables exploratory refinement atop an already stabilized framework, facilitating the discovery of improved solutions.

Beyond these quantitative findings, additional insights were obtained by examining the generated code characteristics

and execution reliability. Here, “lines of code” denote effective lines after excluding blank spaces, while “accuracy” represents the proportion of modules executing without runtime exceptions. Using the N30M20CU20 instance as a case study, population statistics were computed for the four principal modules. As illustrated in Fig. 9 and Table 5, the standard deviation of code length across individuals (std) serves as an indicator of structural variability. For three of the four modules, MLCEMA exhibits the largest std values, suggesting that multi-LLM collaboration yields greater structural diversity than alternative frameworks. Although MA2 achieves the highest execution accuracy due to reliance on the high-performing Gemini-2.5-flash, MLCEMA’s comparatively lower initial correctness does not degrade final outcomes, since the LLM-leader progressively rectifies errors during evolution. These observations indicate that diversity in the initial population exerts a stronger influence on automated algorithm synthesis than early-stage execution correctness. Consequently, effective memetic algorithm design should emphasize coordinated optimization across multiple modules rather than isolated refinement of individual components.

Table 3 Comparison of OES_{mean} values across algorithm variants on benchmark instances.

Instance (N-M-CU)	I-MA	MA1	MA2	LCMA1	LCMA2	LCMA
10-5-6	995	0.129	0.218	0.203	0.222	0.224
10-8-6	925	0.118	0.230	0.200	0.211	0.228
10-8-8	949	0.121	0.231	0.205	0.213	0.228
10-10-6	1001	0.119	0.233	0.205	0.221	0.218
10-10-8	1072	0.119	0.221	0.198	0.227	0.230
15-8-6	1368	0.118	0.228	0.210	0.222	0.229
15-10-8	1262	0.113	0.232	0.198	0.231	0.233
15-10-8	991	0.121	0.230	0.198	0.217	0.228
15-12-8	985	0.115	0.224	0.206	0.222	0.236
15-12-10	953	0.120	0.219	0.200	0.235	0.229
15-12-12	1195	0.113	0.232	0.195	0.230	0.233
15-15-6	2028	0.097	0.222	0.197	0.239	0.252
15-15-8	1692	0.103	0.214	0.190	0.240	0.249
15-15-10	1725	0.098	0.217	0.187	0.246	0.260
20-10-6	2150	0.100	0.220	0.196	0.236	0.240
20-15-6	2053	0.096	0.219	0.199	0.243	0.252
20-15-10	1714	0.095	0.220	0.189	0.249	0.251
20-15-15	1315	0.107	0.216	0.201	0.244	0.246
25-20-10	1857	0.098	0.217	0.180	0.253	0.260
25-20-15	1675	0.113	0.165	0.212	0.258	0.249
30-15-6	2941	0.098	0.198	0.209	0.253	0.255
30-20-6	3089	0.102	0.181	0.200	0.248	0.264
30-20-20	1804	0.107	0.136	0.120	0.275	0.287
50-20-10	3910	0.108	0.136	0.121	0.314	0.320
50-30-10	3919	0.110	0.132	0.110	0.314	0.323
50-30-15	2929	0.135	0.148	0.132	0.280	0.296
50-30-20	2797	0.106	0.133	0.103	0.333	0.332
50-30-25	2383	0.121	0.144	0.121	0.313	0.315
50-40-30	2028	0.101	0.130	0.095	0.334	0.341
50-50-40	1640	0.108	0.134	0.108	0.333	0.331
sum	-	3.308	5.881	5.286	7.657	7.840

Table 4 Elo-based ranking and pairwise performance comparison of algorithm variants

ALGs	A	B	C	D	E	Elo rating (Ablation Drop)
MA1 (A)	-	1	3	0	0	541(-894)

MA2 (B)	29	-	25	7	3	978(-457)
LCMA1 (C)	27	5	-	2	1	752(-683)
LCMA2 (D)	30	23	28	-	6	1284(-151)
MLCEMA (E)	30	27	29	24	-	1435(Baseline)

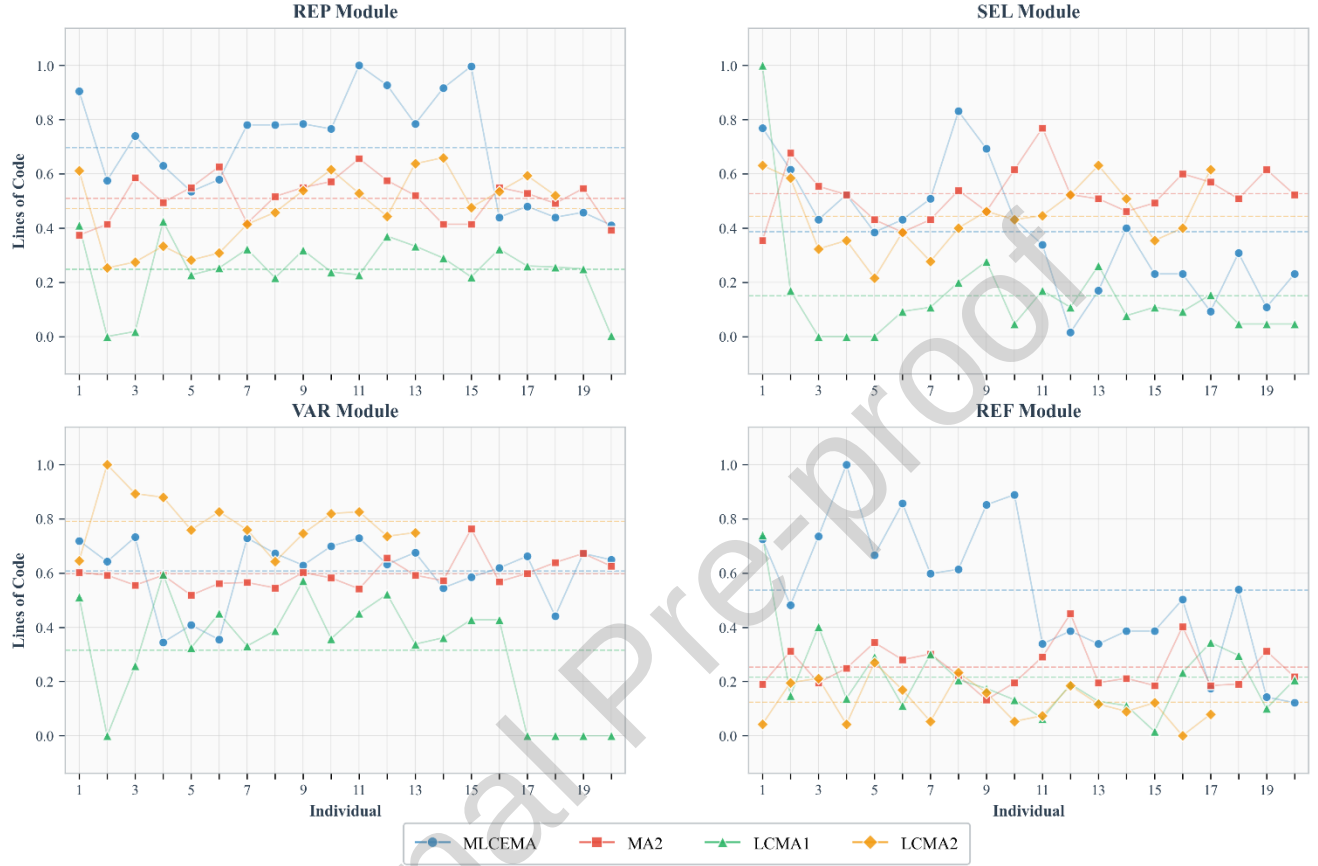


Fig. 9 Individual performance comparison across ablation modules.

Table 5 Statistical results of ablation modules

ALGs	REP Module		SEL Module		VAR Module		REF Module	
	Diversity	Accuracy	Diversity	Accuracy	Diversity	Accuracy	Diversity	Accuracy
MA2	0.079	0.8	0.096	1.0	0.053	0.85	0.079	1.0
LCMA1	0.117	0.55	0.21	0.95	0.2	0.55	0.154	0.75
LCMA2	0.13	-	0.121	-	0.095	-	0.076	-
MLCEMA	0.192	0.95	0.219	0.95	0.121	0.6	0.25	0.6

4.6 Comparison with Other Algorithms

To assess the generalization ability of the proposed MLCEMA framework, comparative experiments were conducted on standard HFJS benchmark datasets. The comparison set includes four high-performing heuristic methods together with the state-of-the-art evolutionary framework EoH [26]. Following the settings in Ref. [45], the selected priority dispatching rules (PDRs) are FIFO, MOPNR, SPT, and MWKR, which can be directly applied to HFJS instances. For EoH, the publicly available implementation was adopted without parameter modification, using $ps = 20$ and $iter = 100$. The MLCEMA framework was independently executed five times, from which the best I-MA and LCMA were retained for evaluation. All competing algorithms were executed ten times to ensure statistical reliability.

Table 6 Performance comparison on HFJS datasets using OES_{mean} and RI_{mean}

Instance	J-M	FIFO	MOPNR	SPT	MWKR	EoH	MLCEMA	RI_{mean} (%)
Brandimarte								
MK01	10-6	0.1692	0.1721	0.1485	0.1634	0.1601	0.1945	11.4512
MK02	10-6	0.1542	0.1607	0.1381	0.1615	0.1611	0.2312	30.2514
MK03	15-8	0.1724	0.1791	0.1342	0.1758	0.1633	0.1825	1.8021
MK04	15-8	0.1625	0.1635	0.1629	0.1718	0.1575	0.1891	9.1852
MK05	15-4	0.1725	0.1728	0.1519	0.1715	0.1516	0.1872	7.8512
MK06	10-15	0.1419	0.1468	0.1775	0.1445	0.1876	0.2093	10.4513
MK07	20-5	0.1572	0.1526	0.1508	0.1539	0.1725	0.2208	22.0124
MK08	20-10	0.1731	0.1764	0.1487	0.1738	0.1582	0.1774	0.5612
MK09	20-10	0.1725	0.1782	0.1408	0.1751	0.1523	0.1888	5.6125
MK10	20-15	0.1671	0.1768	0.1378	0.1751	0.1468	0.2048	13.8515
mean								11.303
Dauzere9-18a								
09a	15-8	0.1671	0.1742	0.1515	0.1746	0.1648	0.1755	0.4612
10a	15-8	0.1605	0.1712	0.1605	0.1709	0.1661	0.1785	3.9512
11a	15-8	0.1651	0.1751	0.1517	0.1752	0.1603	0.1809	3.1624
12a	15-8	0.1685	0.1765	0.1526	0.1692	0.1624	0.1791	1.4215
13a	20-10	0.1626	0.1712	0.1481	0.1714	0.1752	0.1797	2.5012
14a	20-10	0.1654	0.1745	0.1512	0.1772	0.1621	0.1781	0.3621
15a	20-10	0.1684	0.1782	0.1461	0.1805	0.1561	0.1794	-0.6125
16a	20-10	0.1625	0.1722	0.1478	0.1666	0.1753	0.1843	4.9312
17a	20-10	0.1634	0.1748	0.1528	0.1764	0.1626	0.1786	1.2584
18a	20-10	0.1659	0.1765	0.1503	0.1787	0.1577	0.1796	0.4582
mean								1.7895
Behnke								
lar03_1	50-60	0.1435	0.1437	0.2061	0.1472	0.1546	0.2138	3.6412
lar03_2	50-60	0.1332	0.1334	0.1965	0.1367	0.2035	0.2052	0.7012
lar03_3	50-60	0.1391	0.1391	0.2101	0.1413	0.1672	0.2121	0.9451
lar03_4	50-60	0.1373	0.1415	0.1981	0.1421	0.1885	0.2014	1.5952
lar03_5	50-60	0.1429	0.1454	0.2052	0.1441	0.1554	0.2157	4.8812
lar04_1	100-60	0.1448	0.1444	0.2085	0.1456	0.1594	0.2063	-1.1152
lar04_2	100-60	0.1441	0.1426	0.2048	0.1431	0.1683	0.2059	0.4351
lar04_3	100-60	0.1427	0.1423	0.2034	0.1424	0.1735	0.2044	0.3782
lar04_4	100-60	0.1462	0.1441	0.2041	0.1456	0.1652	0.2036	-0.2381
lar04_5	100-60	0.1398	0.1413	0.1994	0.1414	0.1864	0.2005	0.4512
mean								1.1675

The detailed comparison results are listed in Table 6. The MLCEMA framework achieved superior performance on 27 instances overall. On the most favorable instance, the improvement in RI_{mean} reached 30.25%, whereas the largest observed decline was limited to $RI_{mean} = -1.2\%$. In aggregate, the average improvements in RI_{mean} on the MK series, Dauzere9-18a, and Behnke datasets were 11.3%, 1.79%, and 1.17%, respectively. These findings indicate that MLCEMA maintains strong cross-dataset adaptability and demonstrates robust generalization across heterogeneous workshop scheduling scenarios.

For the more challenging HFJSCU scenario, the LCMA evolved by the MLCEMA framework was compared with several advanced optimization methods. Preliminary experiments identified MOPNR as the strongest performer among the PDR strategies for HFJSCU; hence, it was retained for comparison. The competing set further includes the classical GA, the recently proposed population-based evolutionary algorithm AESSPSSO [46], and AutoV [47], which automatically constructs differential evolution operators. In addition, the LLM-assisted LLMMMA [30] and EoH were incorporated. The I-MA solution from Section 4.5 served as the reference baseline for computing OES_i . To ensure fairness, all algorithms with adjustable hyperparameters were configured with $ps = 100$ and $iter = 200$. For AESSPSSO, the cognitive and social coefficients were set to $\beta = 2.05$ and $\gamma = 2.05$, the inertia weight w ranged within $[-10, 10]$, and the search precision was fixed at $prec = 0.001$. LLMMMA adopted $A_p = 7$, $A_g = 15$, $ps = 20$, $n_s = 5$, and $pm = 0.3$, while AutoV retained the operator weight matrix from its original study. EoH used the same parameter settings as in the HFJS experiments, with its evaluation module adapted for HFJSCU; this modified version is denoted MA-EoH. Gemini-2.5-pro was employed for both LLMMMA and MA-EoH. The MLCEMA and LLMMMA frameworks were each executed five independent times, and the best evolved algorithm from these runs was selected for the final evaluation. To ensure a fair and consistent comparison, all competing methods were implemented under the same experimental environment.

Table 7 Friedman test results for all compared algorithms ($\alpha = 0.05$)

ALGs	<i>makespan_{mean}</i>		MOPNR	GA	AESSPSO	AutoV	MA-EoH	LLMMA	LCMA	sum
	rank	<i>p</i> -value								
MOPNR	3.6	0.0000	--	2.4	1.8	3.5	-	-	-	3
GA	5.9		-	--	-	1.2	-	-	-	1
AESSPSO	5.1		-	0.7	--	1.8	-	-	-	2
AutoV	6.8		-	-	-	--	-	-	-	0
MA-EoH	2.6		1.1	3.4	2.8	4.5	--	0.5	-	4
LLMMA	2.8		0.7	3	2.4	4.1	-	--	-	4
LCMA	1.2		2.5	4.8	4.2	5.9	1.5	1.9	--	6

The statistical results are summarized in Table 7. Under a significance threshold of $\alpha = 0.05$, the Friedman test yields a *p*-value below 0.05, confirming that performance differences among the algorithms are statistically significant. To identify pairwise distinctions, the Nemenyi post-hoc test was conducted, producing a critical difference (CD) of 0.5202. When the ranking gap between two algorithms exceeds this CD, their performance disparity is deemed significant. As reported in the rightmost columns of the table, the rank differences between LCMA and all other competitors surpass the CD value, indicating that LCMA achieves statistically superior performance relative to the compared methods.

5 Conclusions

This study introduces the HFJSCU problem to better characterize intelligent manufacturing environments and presents the MLCEMA framework to automate the construction of complex memetic algorithms while reducing reliance on manual expertise. By exploiting the reasoning and program synthesis capabilities of an LLM-leader, an initial algorithmic prototype is generated and subsequently refined through evolution driven by LLM-collaborators, eliminating the need for predefined algorithm templates. Empirical results confirm that contemporary large language models can effectively assume the leader role. Hyperparameter analyses further highlight the stochastic characteristics of LLM-based generation and indicate that enlarging the algorithm population enhances the likelihood of obtaining high-quality solutions. Within the framework, modular decomposition mitigates long-context hallucination by enabling targeted refinement of specific components, while multi-LLM collaboration broadens the search space and promotes structural diversity. The evolutionary mechanism additionally facilitates iterative correction of faulty code, thereby improving computational efficiency. Extensive evaluations on HFJS benchmarks and multi-scale HFJSCU instances demonstrate that MLCEMA achieves strong generalization and exhibits robust automated algorithm design capability.

The conceptual principles underlying MLCEMA provide guidance for subsequent research on LLM-driven optimization systems. In scenario-specific algorithm design, visual prompting can strengthen semantic comprehension, and modular structures can simplify the refinement of complex procedures. Multi-LLM cooperation further enhances diversity and exploratory potential during optimization. Nevertheless, practical deployment may require incorporating additional operational constraints and multiple performance objectives. Future extensions could enable MLCEMA to address dynamic shop-floor conditions, coordinate heterogeneous resources more effectively, and transition from single-objective formulations toward multi-objective optimization settings.

Funding

Supported by the New Generation Information and Communication Technology Innovation Special Project (Grant No. Z251100008125039), the National Natural Science Foundation of China (Grant No. 52575046), and the Open Research Fund of State Key Laboratory of Precision Manufacturing for Extreme Service Performance (Grant No. KFKT2023-10).

CRedit authorship contribution statement

Hongquan Qu wrote the manuscript; Xiaomeng Tong conceived and supervised the study and acquired funding; Shiliang Shao conducted the experiments and performed data analysis; Maolin Cai provided critical feedback; Yan Shi provided technical support; Yushan Ma was responsible for experimental design and resource allocation; Fangwei Ning was in charge of the whole trial. All authors read and approved the final manuscript.

Declaration of Competing Interest

The authors declare no competing financial interests.

Acknowledgements

Not applicable

References

- [1] Agnetis A, Billaut J C, Pinedo M, et al. Fifty years of research in scheduling—Theory and applications. *European Journal of Operational Research*, 2025.
- [2] Zhao S, Zhou H. Learning-driven memetic algorithm for solving integrated distributed production and transportation scheduling problem. *Swarm and Evolutionary Computation*, 2025, 96: 101945.
- [3] Serrano-Ruiz J C, Mula R, Poler R. Development of a multidimensional conceptual model for job shop smart manufacturing scheduling from the industry 4.0 perspective. *Journal of Manufacturing Systems*. 2022, 63: 185–202.
- [4] Li S, Wang R, Zheng P, et al. Towards proactive human–robot collaboration: A foreseeable cognitive manufacturing paradigm. *Journal of Manufacturing Systems*. 2021, 60: 547–552.
- [5] Tamssaouet K, Dauzère-Pérès S, et al. A general efficient neighborhood structure framework for the job-shop and flexible job-shop scheduling problems. *European Journal of Operational Research*. 2023, 311(2): 455–471
- [6] Zhang G, Gao L, Shi Y. An effective genetic algorithm for the flexible job-shop scheduling problem. *Expert Systems with Applications*, 2011, 38(4): 3563–3573.
- [7] Jiang X, Tian W, Liu Y, et al. Energy-efficient scheduling of flexible job shops with complex processes: A case study for the aerospace industry components in China. *Journal of Industrial Information Integration*. 2022, 27: 100293.
- [8] Zhang Q, Shao W, Zhao D, et al. Deep reinforcement learning driven trajectory-based meta-heuristic for distributed heterogeneous flexible job shop scheduling problem. *Swarm and Evolutionary Computation*, 2024, 91: 101753.
- [9] Yao Y, Liu Q, Fu L, et al. A novel mathematical model for the flexible job-shop scheduling problem with limited automated guided vehicles. *IEEE Transactions on Automation Science and Engineering*, 2024, 22: 7449–7462.
- [10] Xie J, Li X, Gao L, et al. A new neighbourhood structure for job shop scheduling problems. *International Journal of Production Research*, 2023, 61(7): 2147–2161.
- [11] Qu H, Tong X, Cai M, et al. Energy-saving scheduling strategy for variable-speed flexible job-shop problem considering operation-dependent energy consumption. *Expert Systems with Applications*, 2024, 256: 124952.
- [12] Du Y, Li J, Li C, et al. A reinforcement learning approach for flexible job shop scheduling problem with crane transportation and setup times. *IEEE Transactions on Neural Networks and Learning Systems*, 2022, 35(4): 5695–5709.
- [13] Lei K, Guo P, Zhao W, et al. A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem. *Expert Systems with Applications*, 2022, 205: 117796.
- [14] Fan C, Wang W, Tian J. Flexible job shop scheduling with stochastic machine breakdowns by an improved tuna swarm optimization algorithm. *Journal of manufacturing systems*, 2024, 74: 180–197.
- [15] Tian Z, Jiang X, Liu W, et al. Energy-efficient scheduling for lot-streaming flexible job-shop considering machine deterioration and maintenance. *Journal of Manufacturing Systems*, 2025, 82: 927–946.
- [16] Li R, Gong W, Lu C, et al. A learning-based memetic algorithm for energy-efficient flexible job-shop scheduling with type-2 fuzzy processing time. *IEEE Transactions on Evolutionary Computation*, 2022, 27(3): 610–620.
- [17] Mustapha K B. A survey of emerging applications of large language models for problems in mechanics, product design, and manufacturing. *Advanced Engineering Informatics*, 2025, 64: 103066.
- [18] Wu X, Wu S, Wu J, et al. Evolutionary computation in the era of large language model: Survey and roadmap. *IEEE Transactions on Evolutionary Computation*, 2024, 29(2): 534–554.
- [19] Huang S, Yang K, Qi S, et al. When large language model meets optimization. *Swarm and Evolutionary Computation*, 2024, 90: 101663.
- [20] Liu S, Chen C, Qu X, et al. Large language models as evolutionary optimizers. 2024 IEEE Congress on Evolutionary Computation (CEC), Yokohama, Japan, 2024: 1–8.
- [21] Shtedritski A, Rupprecht C, Vedaldi A. What does CLIP know about a red circle? Visual prompt engineering for VLMs. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, Paris, France, Oct. 2023: 11987–11997.
- [22] Yang Y, Peng H, Shen Y, et al. Imagebrush: Learning visual in-context instructions for exemplar-based image manipulation. *Advances in Neural Information Processing Systems*, 2023, 36: 48723–48743.
- [23] Cai M, Liu H, Mustikovela S K, et al. ViP-LLaVA: Making large multimodal models understand arbitrary visual prompts. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Seattle, USA, Jun. 2024: 12914–12923.
- [24] Liu D, Yang M, Qu X, et al. A survey of attacks on large vision–language models: Resources, advances, and future trends. *IEEE Transactions on Neural Networks and Learning Systems*, 2025.

- [25] Romera-Paredes B, Barekatin M, Novikov A, et al. Mathematical discoveries from program search with large language models. *Nature*, 2024, 625(7995): 468–475.
- [26] Liu F, Tong X, Yuan M, et al. Evolution of heuristics: towards efficient automatic algorithm design using large language model. *Proceedings of the 41st International Conference on Machine Learning (ICML)*, Vienna, Austria, 2024.
- [27] Ye H, Wang J, Cao Z, et al. Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 2024, 37: 43571–43608.
- [28] Pluhacek M, Kovac J, Viktorin A, et al. Using LLM for automatic evolution of metaheuristics from swarm algorithm SOMA. *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO Companion)*, Melbourne, Australia, 2024: 2018–2022.
- [29] Van Stein N, Bäck T. Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics. *IEEE Transactions on Evolutionary Computation*, 2024, 29(2): 331–345.
- [30] Li R, Wang L, Sang H, et al. LLM-assisted automatic memetic algorithm for lot-streaming hybrid job shop scheduling with variable sublots. *IEEE Transactions on Evolutionary Computation*, 2025.
- [31] An S, Ma Z, Lin Z, et al. Make your llm fully utilize the context. *Advances in Neural Information Processing Systems*, 2024, 37: 62160–62188.
- [32] Pluhacek M, Kazikova A, Kadavy T, et al. Leveraging large language models for the generation of novel metaheuristic optimization algorithms. *Proceedings of the Companion Conference on Genetic and Evolutionary Computation (GECCO Companion)*, Lisbon, Portugal, 2023: 1812–1820.
- [33] Zhao W, Tong X, Cai M. AI for tilting pad bearings: Smart prediction and efficient optimization of tilting pad journal bearings based on deep learning. *Chinese Journal of Mechanical Engineering*, 2026: 100232.
- [34] Yang W, Some L, Bain M, et al. A comprehensive survey on integrating large language models with knowledge-based methods. *Knowledge-Based Systems*, 2025, 318: 113503.
- [35] Li A, Wong P K Y, Tao X, et al. An interactive system for 3D spatial relationship query by integrating tree-based element indexing and LLM-based agent. *Advanced Engineering Informatics*, 2025, 66: 103375.
- [36] Naseem U, Chakraborty T, Chang K W, et al. LLM Alignment should go beyond Harmlessness–Helpfulness and incorporate Human Agency. *Cognitive Computation*, 2026, 18(1): 26.
- [37] He J, Treude C, Lo D. Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 2025, 34(5): 1–30.
- [38] Zhang S, Wang Q, Liu Q, et al. VulTrLM: LLM-assisted vulnerability detection via AST decomposition and comment enhancement. *Empirical Software Engineering*, 2026, 31(1): 1–36.
- [39] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 2022, 35: 24824–24837.
- [40] Chang Y, Wang X, Wang J, et al. A survey on evaluation of large language models[J]. *ACM transactions on intelligent systems and technology*, 2024, 15(3): 1–45.
- [41] Brandimarte P. Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations research*, 1993, 41(3): 157–183.
- [42] Dauzère-Pérès S, Paulli J. An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search. *Annals of Operations Research*, 1997, 70(0): 281–306.
- [43] Fowler J W, Möñch L. A survey of scheduling with parallel batch (p-batch) processing. *European journal of operational research*, 2022, 298(1): 1–24.
- [44] Ojuri S, Han T A, Chiong R, et al. Optimizing text-to-SQL conversion techniques through the integration of intelligent agents and large language models. *Information Processing & Management*, 2025, 62(5): 104136.
- [45] Wang R, Wang G, Sun J, et al. Flexible job shop scheduling via dual attention network-based reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 2023, 35(3): 3091–3102.
- [46] Alimohammadi M, Akbarzadeh-T M R. State-space adaptive exploration for explainable particle swarm optimization. *Swarm and Evolutionary Computation*, 2025, 94: 101868.
- [47] Tian Y, Zhang X, He C, et al. Principled design of translation, scale, and rotation invariant variation operators for metaheuristics. *Chinese Journal of Electronics*, 2023, 32(1): 111–129.

Hongquan Qu, born in 1994, is currently a PhD candidate at *School of Automation Science and Electrical Engineering, Beihang University, China*. His research interests include evolutionary algorithms, evolutionary optimization, and their applications.

Xiaomeng Tong, born in 1989, is currently an associate professor at *School of Automation Science and Electrical Engineering, Beihang University, China*. He received his PhD degree in mechanical engineering from *Texas A&M University, USA*, in 2018. His main research interests include intelligent manufacturing, thermo-elasto-hydro-dynamic analysis of fluid bearings and artificial intelligence.

Shiliang Shao, born in 2002, is currently a master's candidate at the *School of Automation Science and Electrical Engineering, Beihang University, China*. His research interests include workshop scheduling and robotic systems.

Maolin Cai, born in 1972, is currently a professor at *School of Automation Science and Electrical Engineering, Beihang University, China*. He received his PhD degree in mechanical engineering from *Tokyo Institute of Technology, Japan*, in

2002. His main research interests include intelligent manufacturing and compressed air energy storage.

Yan Shi, born in 1981, is currently a professor at *School of Automation Science and Electrical Engineering, Beihang University, China*. He received a Ph.D. degree in mechanical engineering from *Beihang University, China*, in 2012. His research interests include intelligent optimization, mechatronic engineering, and intelligent control.

Yushan Ma, born in 1968, is currently the Director of *Liupanshan Laboratory, China*. He received a Ph.D. degree from *Xi'an University of Technology, China*, in 2009. His research interests include intelligent manufacturing technologies and advanced design of high-end control valves.

Fangwei Ning, born in 1992, is currently an associate professor at *School of Automation Science and Electrical Engineering, Beihang University, China*. He received a Ph.D. degree in mechanical engineering from *Beihang University, China*, in 2021. His research interests include intelligent manufacturing technologies and system design.

Journal Pre-proof